
# DishDive Application: Menu Review Analysis

**Mr.Chayapol Mahatthanachai**
**Mr.Shinathaj Chinskul**

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF SCIENCE (COMPUTER SCIENCE)
SCHOOL OF INFORMATION TECHNOLOGY
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2025

**DishDive Application: Menu review analysis**

Mr.Chayapol Mahatthanachai **B.SC. (Computer Science)**
Mr. Shinathaj Chinskul **B.SC. (Computer Science)**

A Thesis Submitted in Partial Fulfillment of the Requirements for
The Degree of Bachelor of Science (Computer Science)
School of Information Technology
King Mongkut's University of Technology Thonburi
2025

**Thesis Committee**

..... *Chairman of Thesis Committee*
(Asst. Prof. Dr.Chonlameth Arpnikanondt, Ph.D.)

..... *Member and Thesis Advisor*
(Asst. Prof. Dr.Worarat Krathu, Ph.D.)

| | | |
|---|---|---|
| **Thesis Title** | : | DishDive Application: Menu Review Analysis |
| **Thesis Credits** | : | 6 |
| **Candidate** | : | Mr.Chayapol Mahatthanachai |
| | : | Mr.Shinathaj Chinskul |
| **Thesis Advisors** | : | Asst. Prof. Dr.Worarat Krathu, Ph.D. |
| **Program** | : | Bachelor of Science |
| **Field of Study** | : | Computer Science |
| **Department** | : | Computer Science |
| **Faculty** | : | School of Information Technology |
| **Academic Year** | : | 2025 |

## Abstract

In the modern dining experience, diners face a time-consuming and inefficient process of sifting through scattered online reviews across multiple platforms to select a meal. This project addresses this inefficiency by designing and developing DishDive, an AI-powered mobile application that analyzes online customer feedback to recommend top-rated dishes in Bangkok.

The primary objective was to create a centralized system that extracts, processes, and summarizes social media comments to provide actionable, dish-level insights. The methodology involved implementing a comprehensive system architecture, beginning with a web crawling module to gather review data from platforms like Facebook and X. This data was then processed through a pipeline utilizing a Large Language Model (LLM) to perform sentiment analysis and keyword extraction.

The result of this project is a functional DishDive application that provides users with a searchable restaurant database and personalized dish recommendations. The system successfully transforms unstructured, scattered opinions into summarized, data-driven insights, significantly reducing the time users spend on manual searches and enhancing their ability to make informed dining decisions.

# Acknowledgements

The completion of the DishDive capstone project would not have been possible without the guidance, support, and generosity of several key individuals.

First and foremost, the team extends its deepest gratitude to our advisor, Asst. Prof. Dr.Worarat Krathu, for her unwavering dedication, insightful critiques, and expert direction throughout the challenging phases of research, development, and system integration.

We are also profoundly grateful to our Senior Consultant, who, through their connection with our advisor, provided consistent, invaluable support. Her commitment to weekly check-ins, rigorous review, and refinement of the technical components and documentation was instrumental in maintaining the project's quality and focus.

The team also wishes to recognize the essential contributions of three External Technical Consultants who provided crucial support in the final deployment and infrastructure setup, ensuring the application was fully containerized and deployable.

Finally, the team wishes to recognize the foundational support provided by the family. The successful pivot to the high-performance GPT-4o mini model—a critical step that ensured the technical viability of the entire project—was made possible through their financial support. This assistance directly overcame a major technical ceiling and is acknowledged with the utmost sincerity.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Background

In the modern dining experience, selecting a meal at a restaurant is often a time-consuming and frustrating process. Customers rely on online reviews, social media discussions, and scattered opinions, making it difficult to identify the best dishes. The lack of a centralized and structured system for aggregating and analyzing this information results in inefficiencies and uncertainty when making dining decisions [3].

Currently, users must manually sift through multiple sources to determine popular menu items, which can be overwhelming and lead to suboptimal choices. Restaurants and food enthusiasts lack a streamlined solution that objectively compiles customer feedback and ratings into meaningful insights. This inefficiency highlights the need for an intelligent system that simplifies the decision-making process [1].

## 1.2 Objectives

This project aims to develop DishDive, an AI-powered system that analyzes online customer feedback to recommend top-rated dishes in Bangkok. It will extract and process social media comments, perform sentiment analysis, and summarize insights. The system will feature a searchable restaurant database, interactive summary map, and personalized recommendations based on user preferences. A mobile application will ensure a seamless and user-friendly experience.

## 1.3 Scope

**Functions and Features:**

- Extract and process social media comments from platforms such as Facebook, X, and Instagram.
- Perform sentiment analysis and categorize feedback based on food and overall impressions.
- Summarize analysis results to recommend top-rated dishes at selected restaurants.
- Provide a searchable restaurant database with an interactive summary map.
- Support user registration, authentication, and personalized recommendations based on preferences.
- Enable users to view detailed comment analysis for each dish and contribute their own reviews.

**Users:**

- General users seeking restaurant recommendations.

- Registered users with access to preference-based suggestions and review contributions.

**Related Systems:**

- Social media platforms as data sources for extracting customer reviews.
- AI-driven sentiment analysis and keyword extraction models.
- Database management system for storing and retrieving restaurant insights.

**Supported Platforms:**

- Mobile application compatible with iOS and Android.
- Cloud-based backend infrastructure for data processing and storage.

## 1.4 Expected Benefits

The development of DishDive is expected to provide the following benefits:

- Users will have a streamlined tool to quickly identify top-rated dishes based on AI-analyzed customer feedback, reducing the time spent searching through scattered reviews.
- The system will enhance decision-making by summarizing social media comments and sentiment analysis, ensuring users make informed dining choices.
- Restaurants can gain valuable insights into customer preferences, helping them improve their menu offerings and overall service.
- The integration of user preferences will provide personalized recommendations, enhancing the dining experience for individuals with specific tastes.
- The project will contribute to new knowledge in AI-driven sentiment analysis and recommendation systems, particularly in the restaurant industry.

# Chapter 2

# Feasibility Study

## 2.1 Introduction

The feasibility section evaluates the practicality of implementing the proposed system, which extracts and processes social media comments to generate restaurant and dish recommendations. While existing sentiment analysis tools and restaurant recommendation systems provide insights into overall restaurant quality, they often fail to analyze dish-specific feedback, leaving diners uncertain about the best meal choices. Our project addresses these gaps by leveraging AI-driven sentiment analysis, a structured database, and personalized recommendations to deliver precise, dish-level insights. This section will assess the technical, operational, and financial feasibility of the project, ensuring its viability in real-world applications.

## 2.2 Problem Statement

Currently, selecting a meal at a restaurant is a time-consuming and inefficient process due to the scattered nature of customer reviews across multiple online platforms. Users must manually search through various sources such as Facebook, X, and Instagram to identify recommended dishes, leading to information overload, inconsistencies, and difficulty in decision-making.

Additionally, sentiment and keyword analysis are not readily available, making it challenging for users to assess the overall quality and popularity of specific menu items. Restaurants also lack structured insights into customer preferences, limiting their ability to improve their offerings effectively.

Without a centralized system to analyze and summarize customer feedback, users face frustration, delays, and suboptimal dining choices, while restaurants miss opportunities to optimize their menus based on real-time consumer sentiment. The absence of a data-driven approach highlights the need for an AI-powered solution to streamline the process of discovering top-rated dishes.

## 2.3 Related research and projects

Several research studies and projects have explored AI-driven sentiment analysis and recommendation systems, particularly in the food and restaurant industry [5]. However, existing solutions face limitations that DishDive aims to address:

- **Sentiment Analysis for Restaurant Reviews** – Various research projects have utilized Natural Language Processing (NLP) to analyze restaurant reviews, identifying positive and negative sentiments. However, these studies often focus on generic restaurant ratings rather than dish-specific analysis, limiting their usefulness in meal selection [5].
- **Restaurant Recommendation Systems** – Many existing recommendation systems, such as those used by food delivery platforms, suggest restaurants based on overall ratings and location. However,

they do not analyze detailed customer feedback on individual dishes, making it difficult for users to select the best meal at a given restaurant [2].

- **Social Media Data Extraction** – Previous research has explored methods for extracting restaurant-related comments from social media. However, these projects often struggle with handling unstructured data and noise, leading to inaccurate or incomplete analysis [4].

DishDive enhances previous research by focusing on dish-specific recommendations rather than restaurant-level ratings. It integrates AI-driven sentiment analysis and keyword extraction to categorize feedback more effectively. Unlike traditional recommendation systems, DishDive leverages real-time social media data to provide up-to-date insights on trending dishes. Additionally, its user preference features allow for personalized meal suggestions, making the system more interactive and tailored to individual tastes.

## 2.4 Requirement Specifications

Based on the initial evaluation, the implementation of DishDive will aim to satisfy the following requirements:

### Functional Requirement

- Extract, process, and categorize customer comments from social media platforms (Facebook, X, Instagram).
- Perform AI-powered sentiment analysis and keyword extraction to assess customer opinions.
- Summarize and recommend top-rated dishes based on analyzed feedback.
- Provide a searchable restaurant database with an interactive summary map displaying analyzed restaurants in Bangkok.
- Enable user registration and authentication, supporting account management and preference customization.
- Allow users to set food preferences, save favorite dishes, and contribute their own reviews.
- Display detailed comment analysis for each dish, including common associated keywords.
- Offer a mobile application with a user-friendly interface for easy access.

### Non-Functional Requirement

- The system is expected to handle several hundred concurrent users, with scalability for future growth.
- Mobile Application: Android (version 10 and above) and iOS (version 13 and above).
- User-Friendly Interface.

## 2.5 Implementation Plan

The DishDive project implementation is estimated to take approximately 5–6 months, divided into key phases as follows:

## Phase 1: UX/UI Design (2 weeks)

The UX/UI Design phase aims to develop a visually intuitive and user-friendly interface for the DishDive mobile application. The primary objective is to facilitate smooth navigation, establish a clear visual structure, and create an engaging user experience that prioritizes usability and accessibility.

**Research and Wireframing**

- Conduct competitive analysis on existing food recommendation apps to identify best practices.
- Define user personas and user journey mapping based on target audience needs.
- Create low-fidelity wireframes outlining key screens such as:
  - Home screen with personalized recommendations
  - Restaurant search and filtering options
  - Dish details with customer sentiment analysis
  - User profile and preferences settings

**High-Fidelity UI and Prototyping**

- Transform wireframes into high-fidelity mockups using Figma, incorporating branding and color schemes.
- Develop interactive prototypes to replicate user interactions and application flow.

## Phase 2: Database Design (4 weeks)

The Database Design phase aims to establish a well-structured DishDive database that efficiently handles storage, retrieval, and management of restaurant data, user preferences, and AI-generated insights. The primary objective is to ensure scalability, data consistency, and optimized query performance, enabling seamless integration between the backend, AI processing system, and mobile application.

**Data Modeling and Schema Design**

- Define key data entities, including:
  - Users: Account details, preferences, blacklist, and saved dishes.
  - Restaurants: Business name, location, cuisine category, and overall ratings.
  - Dishes: Dish name, associated restaurant, customer sentiment, and relevant keywords.
  - Reviews: Extracted user comments, sentiment analysis scores, and source platform.
- Develop an Entity-Relationship Diagram (ERD) to illustrate data relationships within the system.
- Determine to implement a SQL (MySQL) database based on project requirements.
- Apply database normalization (SQL) to reduce redundancy.
- Establish an indexing strategy to optimize query efficiency for dish recommendation retrieval.

**Implementation and Optimization**

- Set up the database environment (MySQL).
- Establish schemas or tables based on the finalized Entity-Relationship Diagram (ERD).
- Apply data constraints and validation rules to ensure data accuracy and consistency.

- Implement stored procedures or indexing methods to enhance query performance and retrieval speed.
- Conduct testing on data operations, including retrieval, insertion, and updates, to verify seamless backend integration.
- Optimize database performance to support scalability and handle high-concurrency transactions efficiently.

### Phase 3: Backend Preparation (4 weeks)

The Backend Preparation phase is dedicated to configuring the GoLang server and facilitating seamless integration between the database, AI processing module, and mobile application. The primary goal is to establish a scalable, secure, and high-performance backend infrastructure that supports real-time data processing and communication.

**Server Configuration and API Development**

- Set up the GoLang backend environment on a cloud-based platform such as GCP.
- Design the API architecture to manage user interactions, AI-generated recommendations, and database operations.
- Develop RESTful API endpoints for key system functionalities, including:
  - User Authentication & Authorization (Account creation, login, and JWT-based security mechanisms).
  - Restaurant & Dish Information Retrieval (Fetching restaurant details, recommended dishes, and user preferences).
  - Review Processing (Retrieving AI-analyzed sentiment data and extracted keywords).
- Implement middleware to enhance security, input validation, and request handling.
  - Checking JWT tokens (Authentication) and verifying permission (Authorization).
  - Checking required fields (name, email) and ensuring the data types (number).
  - Parsing JSON (body-parser).
- Configure logging and monitoring systems to track API performance and identify potential bottlenecks.

**Database Integration and Performance Testing**

- Establish a connection between the GoLang server and the database (MySQL) while applying query optimization techniques.
- Execute unit testing on API endpoints to validate request processing, response accuracy, and error handling.
- Conduct load testing to evaluate system scalability under high concurrent user requests.
- Enhance backend response time optimization to enable real-time data processing for mobile application users.

### Phase 4: Simple Demo (2 weeks)

The Simple Demo phase aims to build a basic functional prototype of DishDive to showcase its core system capabilities. This phase ensures the proper integration and functionality of the frontend, backend, database, and AI processing modules, serving as an early-stage proof of concept before full-scale development.

**Core Feature Development**

- Establish a test environment for deploying the backend, database, and AI processing system.
- Develop a simplified mobile application interface with essential UI elements, including:
  - Login & Registration screens for user authentication.
  - Restaurant Search & List View displaying restaurant data retrieved from the database.
  - Dish Recommendations Page presenting AI-analyzed dish insights.
- Connect the backend API endpoints to enable data exchange between the mobile app and server.
- Test the basic AI recommendation system, retrieving sentiment-analyzed data for selected dishes.

**Testing and Demo Preparation**

- Perform end-to-end testing to verify seamless communication between the frontend, backend, and database.
- Detect and resolve major performance issues or system bottlenecks affecting data flow.
- Create a simple UI walkthrough to demonstrate key system interactions.
- Finalize a demo version highlighting key features, including restaurant search, AI-driven dish recommendations, and interactive user engagement.

### Phase 5: Web Crawling (14 weeks)

The Web Crawling phase is dedicated to developing and refining the data extraction pipeline to gather restaurant reviews and dish-specific insights from various online sources. This process ensures the system can acquire real-time, structured data to support AI-driven analysis and recommendations.

**Crawler Implementation & Data Integration**

- Determine relevant data sources, including restaurant websites, food review platforms, and social media posts.
- Develop web crawlers using Python-based tools such as Selenium and BeautifulSoup for data extraction.
  - Scraping dynamic web pages.
  - Parsing HTML and XML documents (extracting data from static web pages).
- Implement data filtering techniques to eliminate irrelevant or duplicate content.
- Organize the extracted data into structured formats, categorizing information into:
  - Restaurant details (name, location, type, and ratings).
  - Dish-specific reviews (customer feedback, sentiment analysis, and commonly associated keywords).
- Store the retrieved data in a temporary staging database for further processing.

**Data Refinement & Optimization**

- Apply text-cleaning methods to process extracted data, including the removal of HTML tags, emojis, and unnecessary noise.
- Conduct sentiment analysis preprocessing to refine customer feedback for AI-based review categorization.
  - Text Cleaning (Removing HTML tags).
  - Noise Reduction (Removing stop words).
- Develop an automated scheduling system for periodic data extraction, ensuring continuous updates.
- Perform performance testing to evaluate crawling speed, data accuracy, and system resource consumption.

## Phase 6: LLM Processing (14 weeks)

The LLM Processing phase is dedicated to utilizing Large Language Model (LLM) techniques to analyze customer reviews, extract meaningful insights, and generate structured dish recommendations. This stage ensures that the AI system can efficiently process large-scale textual data, perform sentiment analysis, and detect key trends in user feedback.

**Model Selection & Data Preparation**

- Choose a suitable LLM framework (LangChain, Google Gemini, Google API) based on system requirements.
- Process and clean textual data from the web crawling phase, ensuring it is structured for analysis.
- Instruct the sentiment analysis model to categorize customer feedback into positive, negative, or neutral.
- Apply keyword extraction methods to identify commonly mentioned dish names, flavors, and attributes.

**Model Integration & Performance Tuning**

- Connect the LLM model to the backend to enable real-time AI-powered analysis.
- Implement prompt engineering strategies to improve the accuracy and relevance of generated recommendations.
- Develop response filtering mechanisms to ensure outputs remain unbiased and contextually appropriate.
  - Ensuring output appropriateness (off-topic, irrelevant or inappropriate).
  - Mitigate Bias (societal bias) — detect and neutralize biased content.
- Perform load testing to evaluate the system's ability to handle large volumes of customer reviews.
- Optimize the AI processing pipeline to maintain a balance between computational efficiency and response speed.

## Phase 7: Preference Algorithm (4 weeks)

The Preference Algorithm is dedicated to designing an algorithm for tailoring dish recommendations based on user-defined tastes, dietary restrictions, and blacklisted ingredients. By leveraging structured keyword matching and ranking mechanisms, this system ensures that recommendations align with individual user preferences while maintaining performance efficiency.

**Preference Matching & Data Structuring**

- Design a user preference database that stores selected attributes such as preferred flavors, dietary restrictions, and blacklisted ingredients.
- Preference settings are based on extracted keywords such as ingredient type, flavor profile, and cuisine style. Additionally, users can adjust sentiment score thresholds to prioritize dishes with higher or lower positivity ratings.
- Implement a matching algorithm that assigns priority scores by comparing user preferences with dish attributes.
- Ensure blacklist functionality overrides all other preferences, filtering out dishes containing explicitly unwanted ingredients.

**Ranking System & Recommendation Optimization**

- Establish a weighted scoring model where dishes matching multiple preferences receive higher rankings for a specific user.
- Implement a sorting mechanism to prioritize recommendations from highest to lowest relevance by default, and apply the weighted scores for each individual user.
- Optimize query execution using indexed searches to minimize processing time for preference-based filtering.
- Perform periodic evaluations to fine-tune ranking weights and ensure accurate, user-centric recommendations.

## Phase 8: Frontend Display (4 weeks)

The Frontend Display phase is dedicated to developing and refining the user interface (UI) to provide an engaging, user-friendly, and responsive experience for DishDive users. This phase ensures that AI-driven recommendations, restaurant details, and user preferences are seamlessly integrated into the mobile application for a smooth and intuitive interaction.

**UI Development & Data Integration**

- Implement frontend components to support core functionalities, including:
  - Restaurant search and filtering for personalized discovery.
  - Dish recommendation display featuring AI-analyzed insights.
  - Customer review summaries presenting sentiment trends and keywords.
  - User profile and preference management to enhance personalized suggestions.
- Establish backend API connections to dynamically retrieve and display real-time restaurant, dish, and review data.
- Ensure a responsive design that adapts to different screen sizes and mobile devices.
- Enable real-time UI updates, allowing users to receive live recommendations and modify preferences effortlessly.

**User Experience Enhancement & Testing**

- Improve UI transitions and animations to ensure a fluid and visually appealing user experience.
- Conduct usability testing to evaluate accessibility, ease of navigation, and overall user satisfaction.
- Optimize frontend performance by enhancing load times and refining data-fetching processes.
- Implement error-handling mechanisms to manage API failures and slow response times efficiently.

## Phase 9: Integration & Testing (2 weeks)

The Integration and Testing phase focuses on seamlessly connecting all system components — including the frontend, backend, database, AI processing, and web crawling modules — to ensure they function cohesively. This phase is critical for validating system stability, performance, and reliability before the final deployment.

**System Integration**

- Establish connections between all key components, including the mobile application, backend APIs, AI-driven recommendation engine, and database.
- Ensure smooth data exchange across modules, maintaining real-time updates and synchronization.
- Conduct API validation to confirm that requests and responses function correctly.
- Set up logging and monitoring tools to track system operations and detect potential issues.

**Testing and Performance Optimization**

- Execute functional testing to verify that core features such as search, recommendations, and user preferences, are operating as intended.
- Conduct performance testing to measure response times and optimize database queries.
- Implement security testing to ensure authentication, authorization, and data protection.
  - Prevent Abuse.
  - Minimized load from unauthorized access.
- Identify and resolve bugs, inconsistencies, or system bottlenecks to enhance system efficiency.

## Phase 10: Finalization (2 weeks)

The Finalization phase is dedicated to preparing the system for public release, ensuring all necessary adjustments are made, and finalizing documentation for deployment and user adoption.

**Final Optimizations and Deployment Readiness**

- Apply UI/UX refinements based on testing feedback to enhance user experience.
- Improve AI recommendation accuracy to deliver more relevant dish suggestions.
- Finalize database indexing and caching strategies for optimized performance.
- Verify cloud infrastructure readiness to support stable and scalable operations.

**Documentation and Deployment Execution**

- Complete technical documentation detailing the backend, AI processes, and database structures.

- Deploy the system to live production environments, including cloud servers, Google Play, and the App Store.
- Monitor post-launch system performance and collect user feedback for future enhancements.
- Conduct a post-deployment evaluation to assess initial user experience and system stability.

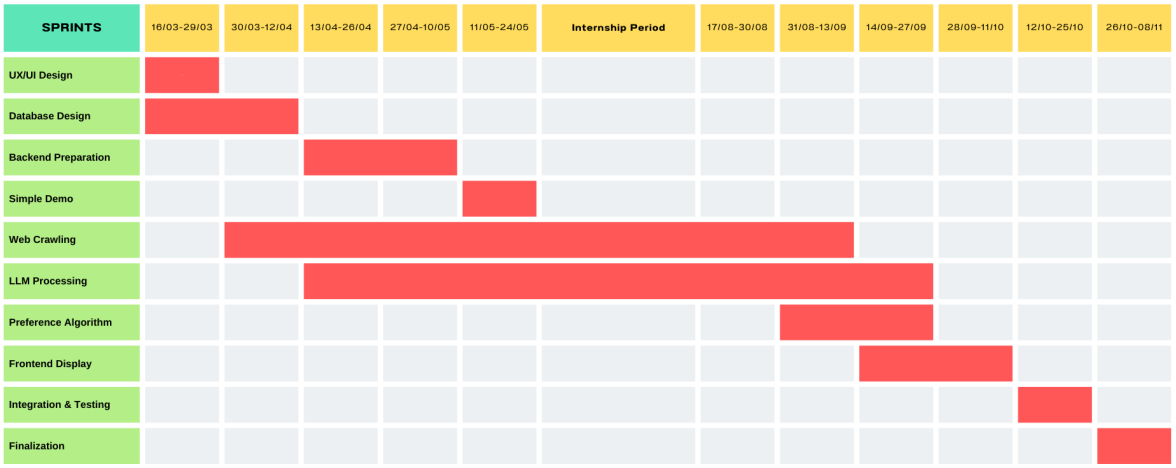


Figure 2.1: DishDive Sprint-Based Gantt Chart

# Chapter 3

# System Analysis and Design

## 3.1 Analysis of the Existing System

The current approach to discovering restaurant dishes heavily relies on manual searches across multiple online platforms, including social media, food blogs, and review websites. Users must sift through scattered and unstructured information, leading to inefficiencies, inconsistencies, and difficulty in identifying the best dishes. The absence of an automated and structured recommendation system creates a time-consuming and often frustrating experience for users.

### 3.1.1 Current System Workflow

**1) Google Reviews** is integrated into the broader Google ecosystem, leveraging Google's robust infrastructure to manage and display user-generated reviews across various services.

- **Data Collection:** User reviews are submitted through Google Maps or Search interfaces.
- **Data Storage:** Reviews are stored in Google's distributed databases.
- **Processing:** Sentiment analysis and spam detection algorithms assess the content of reviews. Machine learning models may be used to prioritize reviews and detect anomalies.
- **Display:** Relevant reviews are displayed on Google Maps and Search results, often sorted by relevance or recency. Businesses can respond to reviews through their Google Business Profile.

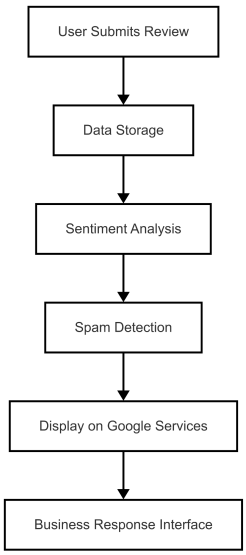


Figure 3.1: Google Reviews System Workflow

**2) Wongnai**, now part of LINE MAN Wongnai, is a Thai platform offering restaurant reviews, food delivery, and lifestyle content. Its architecture supports a wide range of services catering to both users and merchants.

- **User Interaction:** Users can search for restaurants, read and write reviews, and place orders. Merchants can manage their profiles and menus through dedicated interfaces.
- **Backend Services:** Microservices architecture handles different functionalities like search, reviews, and ordering. Services are developed using languages like Java and Go.
- **Data Management:** Data is stored in PostgreSQL databases, with considerations for scalability and performance. Data pipelines are established for analytics and reporting purposes.

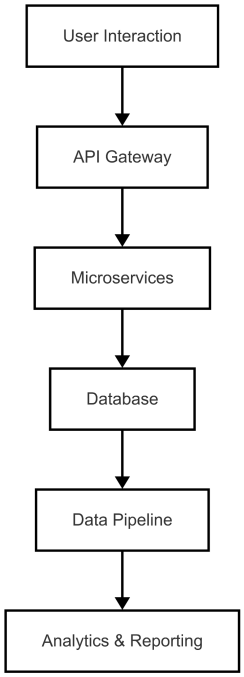


Figure 3.2: Wongnai System Workflow

**3) Tabelog** is a Japanese restaurant review platform that emphasizes user-generated content and rankings. Its architecture supports a large volume of reviews and restaurant data.

- **User Interaction:** Users can browse restaurant listings, submit reviews, and upload photos.
- **Backend Services:** The platform utilizes a combination of Ruby on Rails for the frontend and a custom backend framework. Services are designed to handle high traffic and ensure quick response times.
- **Data Management:** Data is stored in relational databases, with indexing for efficient search capabilities. Caching mechanisms are employed to enhance performance.

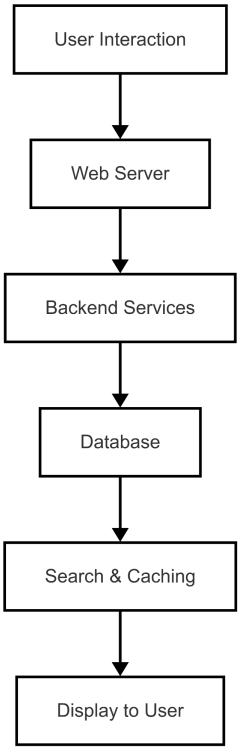


Figure 3.3: Tabelog System Workflow

### 3.1.2 Challenges in the Existing System

**Lack of Personalization**

- **Google Reviews:** Shows generalized restaurant ratings, no tailored dish suggestions.
- **Wongnai:** Offers restaurant suggestions based on location/popularity, not individual taste profiles.
- **Tabelog:** Focuses on chef or location reputation; no personalization of dishes.

**Limited Dish-Level Insights**

- **Google Reviews:** Often tags the restaurant overall; dish mentions are anecdotal.
- **Wongnai:** Some dish mentions, but buried within restaurant-focused pages.
- **Tabelog:** Rarely breaks down sentiment to the individual dish level.

**Time-Consuming Decision-Making**

- **Google Reviews:** Users scroll through long review threads without dish filters.
- **Wongnai:** Requires switching tabs between menus, user reviews, and articles.
- **Tabelog:** Heavy reliance on user judgment to parse subjective, non-sorted dish mentions.

**No Real-Time Data Processing**

- **Google Reviews:** Static once posted; no clear trend tracking for dishes.
- **Wongnai:** Lacks dynamic update features for newly trending dishes.
- **Tabelog:** Restaurant ratings are slow to reflect changes in food quality or trending items.

## 3.2 User requirement analysis

The user requirement analysis aims to identify user expectations, preferences, and key functionalities necessary for an efficient and seamless dining recommendation experience. The requirements were gathered through surveys, interviews, and observations of user interactions with restaurant review platforms and food recommendation systems.

**Primary User Requirements**

**Personalized and Accurate Recommendations** – Users seek AI-powered dish suggestions tailored to their taste preferences, dietary needs, and past selections.

**Real-Time and Reliable Information** – Users require up-to-date, accurate restaurant and dish details, sourced from food blogs, review platforms, and social media.

**Intuitive and User-Friendly Interface** – The mobile application must be easy to navigate, visually appealing, and simple to use for a smooth experience.

**Comprehensive Dish Insights** – Users prefer well-structured summaries that highlight sentiment trends, flavors, and key customer feedback for each dish.

**Advanced Search and Filtering Options** – Users want to search and filter restaurants or dishes based on location, cuisine, and dietary restrictions.

**Efficient System Performance** – The platform must maintain fast response times and smooth operation, even under high user traffic.

**Interactive User Features** – Users desire options to save favorite dishes, submit reviews, and refine recommendations based on their interactions.

These requirements were collected through surveys and interviews with food enthusiasts, restaurant-goers, and online review users, ensuring that DishDive is designed to meet real-world user expectations effectively.

## 3.3 New System Design

### 3.3.1 Site Map

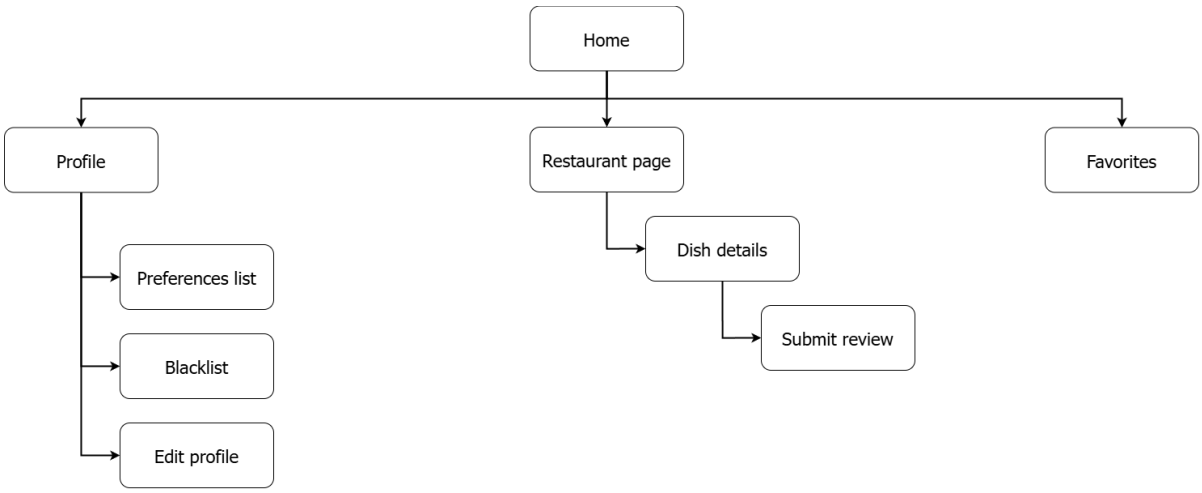


Figure 3.4: Site Map

The site map outlines the navigation structure of the DishDive web application, designed to help users explore food-related data. The home page serves as the central hub, containing the search function, list view and map view toggle, and also linking to other pages including profile, restaurants, and favorites. The profile page offers access to user-specific tools like editing personal information, setting food preferences, and managing a blacklist. The preferences and blacklist pages can also be accessed via the filter modal on the search bar of the home page.

### 3.3.2 Data Processing Data Flow Diagram

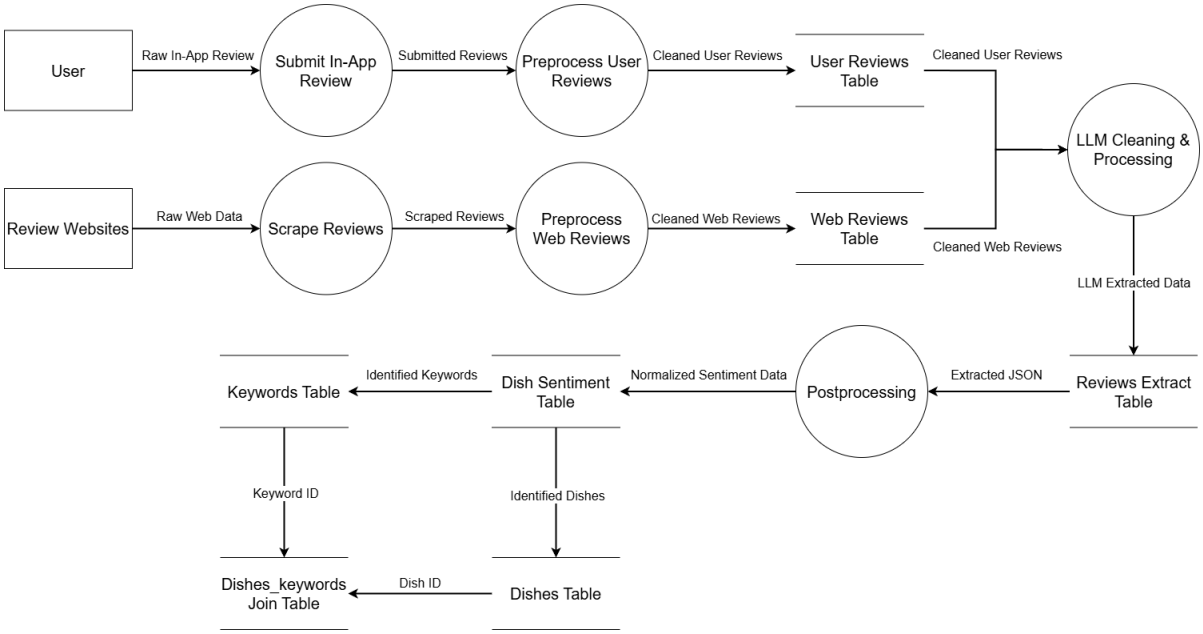


Figure 3.5: Data Processing Data Flow Diagram

This data flow diagram outlines how review data from both external sources and in-app submissions is collected, cleaned, and structured for downstream use.

There are 2 main sources of data, review websites and user reviews submitted directly in-app. Review Websites are scraped periodically to collect user-generated content about restaurants and dishes. This raw text often includes noise like emojis, or inconsistent formatting. User reviews submitted directly in-app go through a parallel but lighter process, since the structure is slightly more predictable.

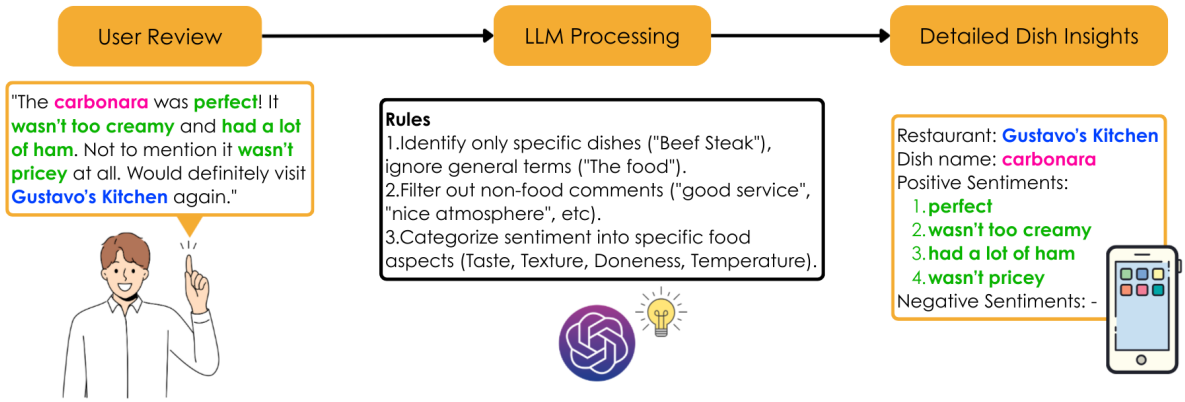


Figure 3.6: Simplified Data Processing Flow

Conceptually, as depicted in the Simplified Data Processing Flow, the system's core function is to take a user-submitted review (from either the in-app form or web scraping), employ the Large Language Model (LLM) with a set prompt to process and structure the data, and generate the final output, which is the structured foundation for a dish's summarized analysis.

**Preprocessing:** This initial stage for both web reviews and user reviews refers to cleaning and formatting the texts. This includes removing redundant sections, splitting paragraphs, stripping unnecessary characters, and removing duplicate content. Both sets of cleaned texts are then stored in separate holding tables, the Web Reviews Table and the User Reviews Table.

**LLM Cleaning and Processing:** This is where the LLM interprets and extracts structured data. Junk reviews (those missing dish names) are discarded. Valid reviews are parsed into structured JSON outputs containing dish names, restaurant names, cuisine, restriction, sentiments, and keywords. These outputs are then logged into the Reviews Extract table as raw extracted data, serving as a staging area.

**Postprocessing:** Following the LLM processing, the data undergoes this crucial stage to transform the raw extracted JSON data from the Reviews Extract table into normalized, row-based data suitable for the relational database. During postprocessing, the system performs entity identification and resolution. Extracted restaurant names and dish names are matched against existing, canonical records in the Restaurants and Dishes tables (including the Dishes_alias table). If a restaurant or dish is encountered for the first time, a new, canonical record is created in the respective master table, and a unique ID is generated by the database system. This ensures that all entities in the database are uniquely identified and consistent. Similarly, new keywords identified during the LLM process are added to the Keywords table, which also generates unique keyword_ids.

**Final Storage:** The primary output of the Postprocessing stage is stored in the Dish Sentiment Table. This table captures the granular sentiment analysis results, linking them directly to the newly resolved or created canonical dish_id and res_id. Each relevant piece of the structured data from the Dish Sentiment is then split and sent to the Keywords table and the Dishes table. Finally, the relationships between these processed dishes and keywords, identified during analysis, are established and recorded in the Dishes_Keywords Join Table.

Once the data is normalized and stored across the relational tables (including the Dish Sentiment Table, Keywords Table, and the Dishes_Keywords Join Table), the cycle is complete, and the data is ready for the client application. When a user performs a search on the DishDive mobile application, the GoLang backend server queries these final tables. The server uses the structured sentiment scores and keywords to aggregate a summarized analysis for each dish. This summarized analysis, which includes each restaurant's top-rated dishes and their corresponding sentiment overviews, is then transmitted back to the mobile application. This ensures that the user is presented with accurate, real-time, and data-driven insights, effectively streamlining their dining decision-making process as intended by the project's objective.

### 3.3.3 Search & Display Process UML Activity Diagram

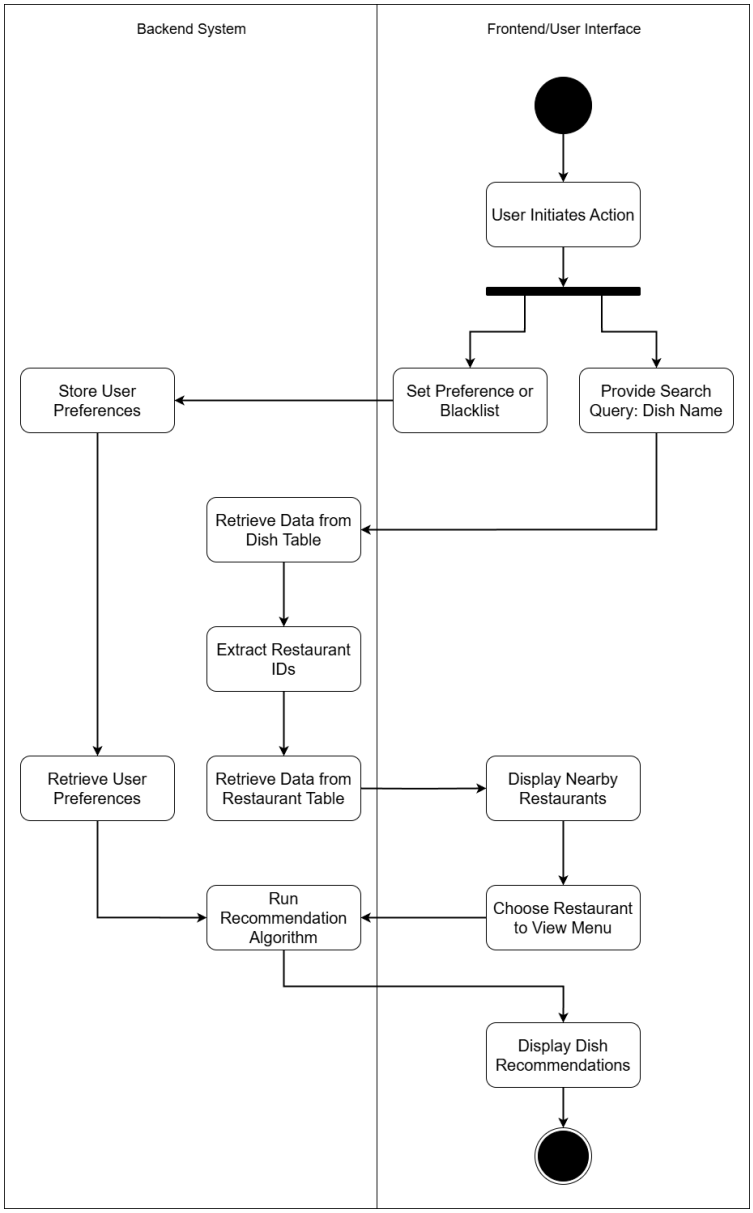


Figure 3.7: Search & Display Process UML Activity Diagram

This UML activity diagram illustrates the process of a user searching for a dish and receiving restaurant and dish recommendations.

The process begins with a user inputting a dish name as a search query. The system queries the Dishes table to find information related to the entered dish name. From the results obtained from the Dishes table, the system extracts relevant restaurant IDs associated with the dish.

These extracted restaurant IDs are then used to query the Restaurant table to retrieve details about the restaurants. Based on the retrieved restaurant information, nearby restaurants that serve the searched dish will be displayed.

The user can choose restaurants from the list, then go into the chosen restaurant's page to view the dish listing menu. Finally, the system displays dish recommendations to the user, which are tailored based on their search, preferences, and related dish data.

Optionally, the user could set the preferences and blacklist before choosing a restaurant. The decision data is stored in the Preferences table, which will be used in the recommendation algorithm.

### 3.3.4 Recommendation Algorithm Flow Chart

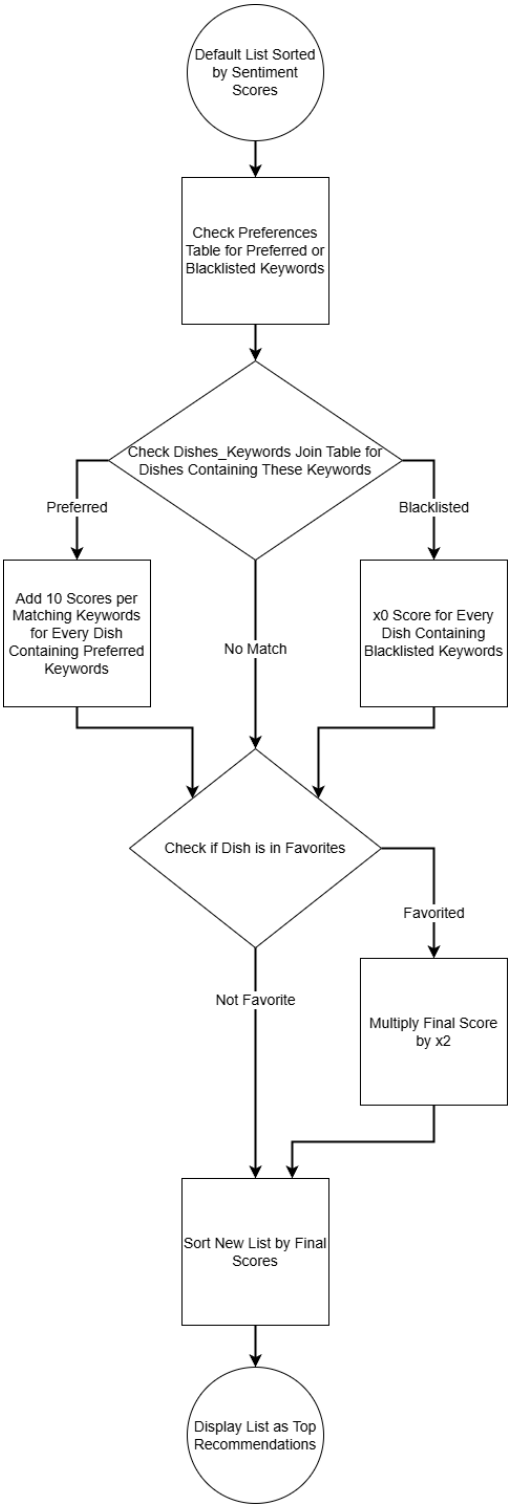


Figure 3.8: Recommendation Algorithm Flow Chart

This flow chart details the step-by-step process of the recommendation algorithm, which refines a default list of dishes into personalized top recommendations for the user.

The process begins with a default list of dishes sorted by sentiment scores. This list represents an initial ranking of dishes based on their inherent sentiment scores. Next, the algorithm checks the preferences table for preferred or blacklisted keywords. This step involves accessing the Preferences table.

A critical decision point follows, where the system checks the Dishes_Keywords join table for dishes containing preferred or blacklisted keywords. If a dish contains preferred keywords, its score is boosted by adding 10 scores per matching keyword. If a dish contains blacklisted keywords, its score is multiplied by 0, effectively removing it from consideration. If no match is found for either preferred or blacklisted keywords, the dish's score remains unchanged from this step. All dishes proceed to the next stage after these score adjustments. For the sentiment score settings, plus 10 if a dish's sentiment score is higher than the set preferred value, and multiply by 0 if a dish's sentiment score is below a set blacklist value.

The algorithm then performs another crucial decision, checking if the dish is in the Favorites table or not. If a dish is in the Favorites table, its final score is multiplied by x2, giving it a significant boost. If the dish is not in the Favorites table, its score is not affected by this step.

After all these scoring adjustments, the algorithm proceeds to sort the new list by final scores. This re-ranks all dishes based on their accumulated scores. Finally, the algorithm concludes by displaying the list as top recommendations to the user, presenting a personalized and refined selection of dishes.

### 3.3.5 ER Diagram

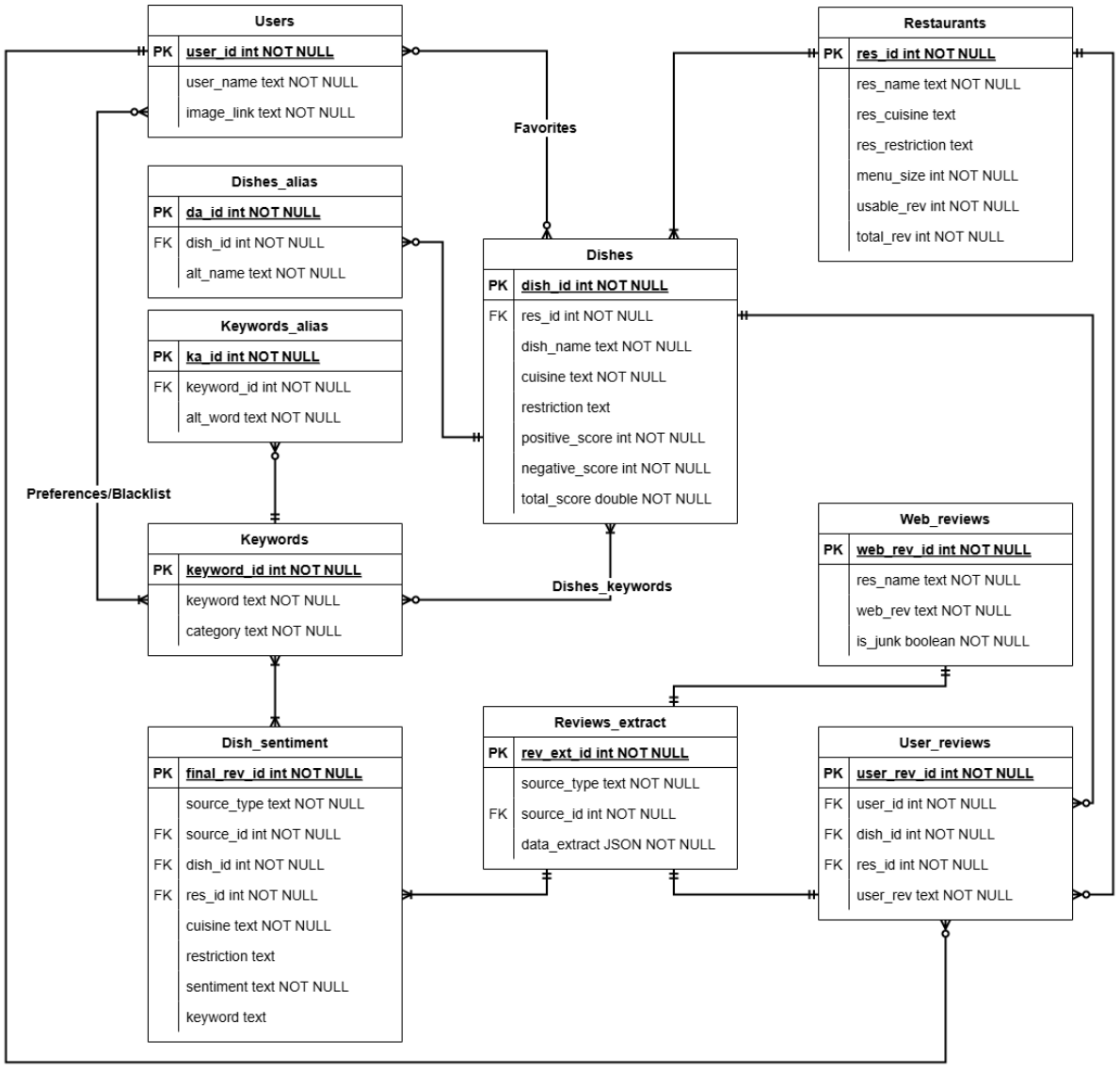


Figure 3.9: ER Diagram

This ER Diagram illustrates the database schema designed to support a dish search and recommendation system, focusing on users, dishes, restaurants, reviews, and personalized preferences.

**1. Users:** Stores information about registered users of the system.

*user_id* (Primary Key, INT, NOT NULL): Unique identifier for each user.

*user_name* (TEXT, NOT NULL): The user's display name or username.

*image_link* (TEXT): A URL or path to the user's profile image.

**2. Restaurants:** Contains details about various restaurants.

*res_id* (Primary Key, INT, NOT NULL): Unique identifier for each restaurant.

*res_name* (TEXT, NOT NULL): The name of the restaurant.

*res_cuisine* (TEXT): The type of cuisine the restaurant serves (e.g., Italian, Thai).

*res_restriction* (TEXT): Any dietary restrictions or special features associated with the restaurant (e.g., Vegan, Halal).

*menu_size* (INT, NOT NULL): The total number of items available on the restaurant's menu.

*usable_rev* (INT, NOT NULL): The count of reviews deemed "usable" or relevant for the restaurant.

*total_rev* (INT, NOT NULL): The total count of all reviews received for the restaurant.

**3. Dishes:** Stores information about individual dishes, serving as the central entity for recommendations.

*dish_id* (Primary Key, INT, NOT NULL): Unique identifier for each dish.

*res_id* (Foreign Key, INT, NOT NULL): Links to the Restaurants table, indicating which restaurant serves this dish.

*dish_name* (TEXT, NOT NULL): The name of the dish.

*cuisine* (TEXT): The cuisine type of the dish

*restriction* (TEXT): Any dietary restrictions or features specific to the dish.

*positive_score* (INT, NOT NULL): A score reflecting positive sentiment towards the dish.

*negative_score* (INT, NOT NULL): A score reflecting negative sentiment towards the dish.

*total_score* (DOUBLE, NOT NULL): An overall aggregated sentiment score for the dish.

**4. Dishes_alias:** Manages alternative names or common misspellings for dishes to improve searchability.

*da_id* (Primary Key, INT, NOT NULL): Unique identifier for each dish alias record.

*dish_id* (Foreign Key, INT, NOT NULL): Links to the Dishes table, indicating which dish this alias belongs to.

*alt_name* (TEXT, NOT NULL): An alternative name for the dish.

**5. Keywords:** Stores a master list of all keywords used in the system for categorization, preferences, and sentiment analysis.

*keyword_id* (Primary Key, INT, NOT NULL): Unique identifier for each keyword.

*keyword* (TEXT, NOT NULL): The actual keyword (e.g., "spicy", "cheap", "delicious").

*category* (TEXT): A broader category for the keyword (e.g., "taste", "cost", "others").

**6. Keywords_alias:** Manages alternative words or phrases for keywords to enhance matching and search capabilities.

*ka_id* (Primary Key, INT, NOT NULL): Unique identifier for each keyword alias record.

*keyword_id* (Foreign Key, INT, NOT NULL): Links to the Keywords table, indicating which keyword this alias belongs to.

*alt_word* (TEXT, NOT NULL): An alternative word or phrase for a keyword.

**7. Favorites (Join Table):** Represents the many-to-many relationship between Users and Dishes that a user has explicitly marked as a "favorite". It tracks only the favorited relationships.

*user_id* (Foreign Key, INT, NOT NULL): Links to the Users table.

*dish_id* (Foreign Key, INT, NOT NULL): Links to the Dishes table.

**8. Dishes_keywords (Join Table):** Represents the many-to-many relationship between Dishes and Keywords, indicating which keywords are associated with which dishes.

*dish_id* (Foreign Key, INT, NOT NULL): Links to the Dishes table.

*keyword_id* (Foreign Key, INT, NOT NULL): Links to the Keywords table.

*frequency* (INT, NOT NULL): Tracks how many times a particular keyword has been associated with a specific dish, used to calculate top keywords.

**9. Preferences/Blacklist (Table):** Stores user-specific preferences and blacklisted keywords, allowing for personalized recommendations.

*user_id* (Foreign Key, INT, NOT NULL): Links to the Users table.

*keyword_id* (Foreign Key, INT, NOT NULL): Links to the Keywords table.

*preference* (FLOAT): A numerical value (usually 0 or 1) indicating a positive preference for the keyword.

*blacklist* (FLOAT): A numerical value (usually 0 or 1) indicating a blacklisted status for the keyword.

**10. Web_reviews:** Stores raw review data scraped or imported from external web sources.

*web_rev_id* (Primary Key, INT, NOT NULL): Unique identifier for each web review.

*res_name* (TEXT, NOT NULL): The restaurant name as it appeared in the web review.

*web_rev* (TEXT, NOT NULL): The full text content of the web review.

*is_junk* (BOOLEAN, NOT NULL): A flag indicating if the review is deemed as junk or not. If the review doesn't mention any dish names, then it will be considered junk.

**11. User_reviews:** Stores reviews written directly by users of the system.

*user_rev_id* (Primary Key, INT, NOT NULL): Unique identifier for each user review.

*user_id* (Foreign Key, INT, NOT NULL): Links to the Users table, indicating who wrote the review.

*dish_id* (Foreign Key, INT, NOT NULL): Links to the Dishes table, indicating which dish this review is for.

*res_id* (Foreign Key, INT, NOT NULL): Links to the Restaurants table, indicating which restaurant this dish review is for.

*user_rev* (TEXT, NOT NULL): The full text content of the user's review.

**12. Reviews_extract:** Stores extracted or processed data points from various review sources (both web and user reviews).

*rev_ext_id* (Primary Key, INT, NOT NULL): Unique identifier for each extracted review.

*source_type* (TEXT, NOT NULL): Indicates the original source type of the review.

*source_id* (Foreign Key, INT, NOT NULL): Links to the specific review ID in its source table (e.g., web_rev_id from Web_reviews or user_rev_id from User_reviews).

*data_extract* (JSON, NOT NULL): Stores structured data extracted from the review content in JSON format.

**13. Dish_sentiment:** Stores the results of sentiment analysis performed on review extracts, associating sentiment with specific dishes and restaurants.

*final_rev_id* (Primary Key, INT, NOT NULL): Unique identifier for each sentiment record.

*source_type* (TEXT, NOT NULL): Indicates the original source type of the review (web or user)

*source_id* (Foreign Key, INT, NOT NULL): The ID of the original review from its source

*dish_id* (Foreign Key, INT, NOT NULL): Links to the Dishes table.

*res_id* (Foreign Key, INT, NOT NULL): Links to the Restaurants table.

*cuisine* (TEXT): The cuisine type.

*restriction* (TEXT): Dietary restrictions.

*sentiment* (TEXT, NOT NULL): The determined sentiment (e.g., 'positive', 'negative', 'neutral').

*keyword* (TEXT): Keywords extracted during sentiment analysis that contributed to the sentiment.

# Chapter 4

# System Functionality

## 4.1 System Functionality

The DishDive application is designed to address consumer decision fatigue in food choice by synthesizing and structuring raw menu review data. The system's primary function is to transform unstructured textual reviews into actionable, personalized, and easy-to-digest menu insights.

The core functionalities provided by the DishDive system include:

### 4.1.1 User Management and Personalization

The application provides standard user account management, enabling secure registration and login via JSON Web Tokens (JWT). Crucially, the system supports user-driven personalization, allowing users to save restaurants to a Favorites list. This data contributes to personalized content filtering and recommendation logic.

### 4.1.2 Restaurant and Menu Browsing

Users can efficiently search and browse the complete catalog of restaurants and their corresponding menus. This functionality supports location-based searches and detailed views, allowing users to inspect individual menu items and their associated review analyses before making a dining decision.

### 4.1.3 Review Submission and Real-Time AI Extraction

This is a critical function that leverages the system's core novelty. Users can submit new reviews directly through the mobile application. Upon submission, the API server immediately triggers an AI-powered extraction process to analyze the review text. This real-time analysis identifies key attributes, sentiments, and topic clusters related to the review, ensuring the latest user feedback is instantly incorporated into the recommendation signals.

### 4.1.4 Menu Review Analysis and Synthesis

The central functionality of DishDive involves presenting users with a summary of menu reviews rather than a stream of raw text. The system analyzes the normalized data to provide insights such as:

- The most frequently mentioned flavor profiles for a dish (e.g., 'Spicy,' 'Sweet,' 'Umami').
- Sentiment breakdown (Positive/Negative) for specific components (e.g., 'The service was great, but the rice was undercooked').
- Summarized recommendation signals derived from both batch-ingested and real-time user reviews.

## 4.2 System Architecture

The DishDive application employs a modern three-tier architecture designed for high scalability, real-time data processing, and robust integration of Artificial Intelligence (AI) services. This structure separates presentation, application logic, and data management into distinct, interconnected modules.

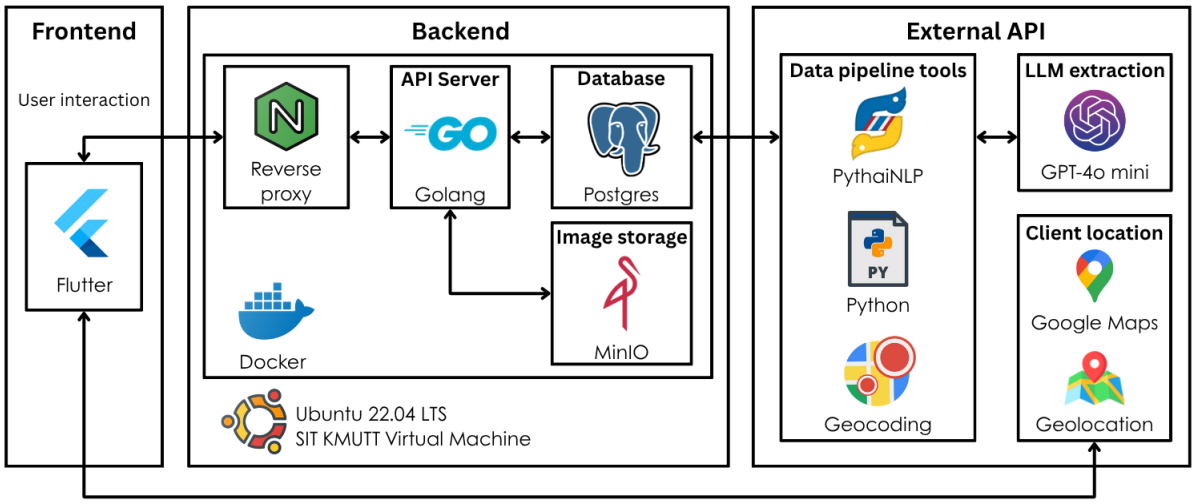


Figure 4.1: System Architecture Diagram

### 4.2.1 High-Level Overview

The system architecture is organized into three distinct, interconnected layers: the Client App, the API Server, and the Storage & AI Backbone. The Client App, built with Flutter/Dart, manages the user interface, state, and all user interactions. This layer communicates over HTTP with the API Server. The API Server, developed in Go using the Fiber web framework and GORM for database operations, houses the core application logic, handles authentication, and integrates the real-time Large Language Model (LLM) services. The third layer, the Storage & AI Backbone, is where data persistence and heavy-duty processing occur, utilizing PostgreSQL for structured data storage (reviews, user profiles) and MinIO for media assets. Additionally, Python scripts are designated as Batch Tools for offline tasks like large-scale data ingestion and cleaning. The core runtime flow sees the Flutter application making an HTTP request to the Go API, which then interacts with either the PostgreSQL database or the OpenAI LLM services to fulfill the request.

### 4.2.2 Architecture Modules and Technology Stack

**A. Client Application (Flutter/Dart)**

The client application is built using Flutter/Dart, enabling a single codebase for Android, iOS, and Web deployment.

- State Management: Utilizes the Provider package for predictable state flow.
- Networking: All API communication is managed via the Dio library.
- User Experience: Includes integration with Maps_flutter and geolocator for map and location services, essential for finding local restaurants.

**B. API Server (Go)**

The API server, developed in Go, acts as the central hub, providing lightweight and high-performance business logic.

- Web Framework: Fiber v2 is used for route handling and request processing.
- Database Interaction: GORM provides the Object-Relational Mapping (ORM) layer for secure and efficient database operations with PostgreSQL.
- Authentication: Handles user authorization using JWT (v4) tokens.
- Real-Time LLM Integration: A key feature is the native HTTP client integration with the OpenAI API. This allows the Go server to perform real-time AI-powered review text extraction immediately after a user submits a review, without relying on slow inter-process communication (e.g., Python subprocesses).

**C. Storage and AI Backbone**

- Database: PostgreSQL serves as the primary data store, hosting all structured data, including user profiles, restaurant listings, raw reviews, and the structured AI-extracted review details (stored in the review_extracts table).
- Object Storage: MinIO, an S3-compatible service, is used for storing media assets like restaurant and menu item images.
- Batch Processing Tools (Python): Python scripts (utilizing libraries like Pandas, psycopg2, and the OpenAI Python client) are designated for offline tasks, such as initial batch ingestion of scraped web reviews and subsequent large-scale data cleaning and processing.

### 4.2.3 Critical Data Flows

**Review Submission and AI Extraction Flow:**

- Client Request: The Flutter application sends a POST /SubmitReview request containing the review text.
- Go API Processing: The Go API server saves the raw review text to PostgreSQL.
- LLM Execution: The Go server's native HTTP client calls the OpenAI API, passing the review text for structured extraction (sentiment, keywords, entity identification).
- Data Storage: The structured LLM output (the review extracts) is written to the review_extracts table.
- Normalization: A background process maps these extracts into the main domain tables, updating aggregate recommendation signals used by the browsing functionality.

**Authentication Flow:**

- Login: The client sends credentials, the Go API verifies them, and returns a JWT.
- Token Provider: The client's TokenProvider stores the JWT.
- Authorized Requests: All subsequent requests (e.g., GET /GetCurrentUser) include the token in the Authorization header, which the Go API verifies before granting access.

## 4.3 Test Plan

The testing strategy for the DishDive application focused on ensuring end-to-end functionality for all critical system components, from the user interface down to the AI processing layer. Given the nature of a functional prototype, testing was performed using manual execution of key user scenarios (often referred to as 'smoke tests' or 'clicking run') to directly confirm that the implementation satisfied the project's assessable features (F1-F7).

### 4.3.1 Distinct Project Features Test

Table 4.1: Distinct Project Features Test (F1–F2.2)

| ID | Feature Description | Test Scenario |
|---|---|---|
| F1 | Analysis on restaurant menus based on online comments using LLM. | Execute batch processing of raw review data via Python script and confirm structured data is written to the PostgreSQL `review_extracts` table. |
| F1.1 | Extract and process social media comments. | Simulate the initial data pipeline by running the web crawler/ingestion script. |
| F1.2 | Analyze sentiment and identify common keywords. | Review the `review_extracts` table for a sample dish and confirm that sentiment and keyword fields are correctly populated by the LLM service. |
| F2 | Summarize analysis results and recommend the best dishes. | Navigate to a specific restaurant and verify that the UI displays the summary and the top-rated dishes based on the processed data. |
| F2.1 | List restaurants in Bangkok and visualize on a summary map. | Load the main map view and confirm that the client application correctly displays the locations of all available restaurants. |
| F2.2 | Provide a restaurant overview with key menu item insights. | View a restaurant's dedicated page and check that the analyzed menu items show accurate, derived insights (e.g., flavor profiles, service comments). |

### 4.3.2 Support Project Features Test

Table 4.2: Support Project Features Test (F3–F7)

| ID | Feature Description | Test Scenario |
|---|---|---|
| F3 | Display a list of restaurants. | Load the main list view and ensure the page is populated with restaurant entities from the database. |
| F4 | Allow users to search for restaurants by name and location. | Use the search bar to filter the restaurant list by name and by proximity/location. |
| F5 | Allow users to save favorite restaurants. | Log in, select a restaurant, tap the "Favorite" button, and confirm it appears in the "Favorites" tab. |
| F6 | Allow users to log in/register. | Execute a full registration process followed by a successful login using the new credentials. |
| F7 | Deploy a functional, scalable application. | Confirm that all application components (Flutter, Go, Postgres, MinIO) are running correctly within the containerized environment. |

## 4.4 Test Results

The testing phase confirmed the correct operation of all assessable features outlined in the project profile.

### 4.4.1 Distinct Project Features Results

Table 4.3: Distinct Project Features Results (F1–F2.2)

| ID | Feature Description | Test Scenario | Result |
|---|---|---|---|
| F1 | Analysis on restaurant menus based on online comments using LLM. | Execute batch processing of raw review data via Python script and confirm structured data is written to the PostgreSQL `review_extracts` table. | Test successful. |
| F1.1 | Extract and process social media comments. | Simulate the initial data pipeline by running the web crawler/ingestion script. | Test successful. |
| F1.2 | Analyze sentiment and identify common keywords. | Review the `review_extracts` table for a sample dish and confirm that sentiment and keyword fields are correctly populated by the LLM service. | Test successful. |
| F2 | Summarize analysis results and recommend the best dishes. | Navigate to a specific restaurant and verify that the UI displays the summary and the top-rated dishes based on the processed data. | Test successful. |
| F2.1 | List restaurants in Bangkok and visualize on a summary map. | Load the main map view and confirm that the client application correctly displays the locations of all available restaurants. | Test successful. |
| F2.2 | Provide a restaurant overview with key menu item insights. | View a restaurant's dedicated page and check that the analyzed menu items show accurate, derived insights (e.g., flavor profiles, service comments). | Test successful. |

### 4.4.2 Support Project Features Results

Table 4.4: Support Project Features Results (F3–F7)

| ID | Feature Description | Test Scenario | Result |
|---|---|---|---|
| F3 | Display a list of restaurants. | Load the main list view and ensure the page is populated with restaurant entities from the database. | Test successful. |
| F4 | Allow users to search for restaurants by name and location. | Use the search bar to filter the restaurant list by name and by proximity/location. | Test successful. |
| F5 | Allow users to save favorite restaurants. | Log in, select a restaurant, tap the "Favorite" button, and confirm it appears in the "Favorites" tab. | Test successful. |
| F6 | Allow users to log in/register. | Execute a full registration process followed by a successful login using the new credentials. | Test successful. |
| F7 | Deploy a functional, scalable application. | Confirm that all application components (Flutter, Go, Postgres, MinIO) are running correctly within the containerized environment. | Test successful. |

# Chapter 5

# Summary and Suggestions

## 5.1 Project Summary and Conclusion

The DishDive application successfully met its core objective: to leverage Artificial Intelligence (AI) to transform overwhelming, unstructured restaurant reviews into concise, actionable menu insights for users facing decision fatigue. By synthesizing sentiment and keywords extracted by a Large Language Model (LLM), the system provides summarized recommendations, ensuring a quick and confident dining choice.

The final system is a robust, three-tier functional prototype built with a modern stack (Flutter, Go, PostgreSQL). All seven assessable features (F1 through F7) were implemented and successfully validated during the testing phase, proving the viability of the AI-driven menu analysis concept. The project delivered a scalable, maintainable application that achieved its stated goal of simplifying the restaurant dining experience.

## 5.2 Problems Encountered and Solutions

Throughout the development lifecycle, two primary technical challenges arose, requiring significant adaptation in the data acquisition and AI processing phases.

### 5.2.1 Web Scraper and Data Acquisition

The process of acquiring a sufficient volume of high-quality, relevant restaurant reviews proved more complex than initially planned. While the web scraping module performed reliably for publicly accessible Google Maps reviews, challenges arose when integrating with local review platforms, specifically Wongnai. The platform's architecture required a dynamic scraping approach.

**The Challenge and Solution:**

The initial automated web scraping script failed to reliably collect data from shops with a high volume of reviews (>8 reviews) due to protective measures and complex dynamic loading. To meet the necessary data quota for the proof of concept, a dual-strategy approach was implemented:

- Automated Scraping: Used for shops with fewer than 8 reviews.
- Manual-Assisted Scraping: Required for shops with 8 or more reviews.

This required a significant manual effort. Approximately 340 shops required manual intervention, a process which ultimately took three full days to complete. This effort was critical to ensuring the database was populated with the required data density for meaningful LLM analysis.

### 5.2.2 Large Language Model (LLM) Selection

The core novelty of the project depended entirely on the ability of an LLM to reliably extract structured data (sentiment, keywords, entity identification) from noisy, real-world review text based on a complex prompt.

**The Challenge and Solution:**

Initial attempts utilized a locally hosted, open-source model, Ollama Qwen, in an effort to maintain full cost control. It quickly became apparent that this model was not powerful enough to handle the complexity and nuance required by the project's extraction prompt. The model consistently failed to produce accurate, structured JSON output, hitting a ceiling that halted development progress.

The solution involved a critical and immediate pivot to a commercial, cloud-based solution: the GPT-4o mini model. This switch was successful instantly, as the higher capability model was able to interpret the complex prompt and deliver the required structured output consistently, validating the project's core hypothesis and allowing the team to quickly finalize the API and application integration.

## 5.3 Suggestions for Further Development

The successful completion of the DishDive prototype lays a strong foundation for future expansion. The following suggestions are proposed for further development (DishDive 2.0):

### 5.3.1 Enhanced Personalization and Recommendation Engine

The current system relies primarily on aggregate sentiment. Future development should incorporate a collaborative filtering engine to provide truly personalized dish recommendations. This would involve tracking user history and preferences to suggest dishes that similar users have enjoyed, moving beyond simple top-rated summaries.

### 5.3.2 Broader Data Acquisition and Scaling

The current system focuses solely on the Bangkok area. Future work should focus on:

- Geographic Expansion: Incorporating restaurants and local review platforms from other major cities and countries.
- Live Social Media Feeds: Expanding the web scraper to handle real-time integration with platforms like X (formerly Twitter) and Instagram to capture the very latest food-related trends and comments.

### 5.3.3 LLM Pipeline Optimization

While the switch to GPT-4o mini resolved the core functionality issue, the cost model is commercial. Future optimization should explore implementing an LLM cascading strategy:

- Pre-filter/Simple Tasks: Route simple sentiment analysis and text cleaning tasks to a cheaper, smaller model (like a specialized fine-tuned model or a cost-effective open-source option).
- Complex Extraction: Reserve the powerful GPT-4o mini only for the most complex entity extraction and summarization tasks. This approach would significantly reduce operational costs while maintaining high quality.

# References


[1] Graham, D. J., Orquin, J. L., & Visschers, V. H. M. (2017). The influence of time of day on decision fatigue in online food choice. *British Food Journal*, 118(3), 735–748. https://doi.org/10.1108/bfj-05-2016-0227

[2] Greene, D., Nguyen, M., & Dolnicar, S. (2024). How do you choose your meal when you dine out? A mixed methods study in consumer food-choice strategies in the restaurant context. *Appetite*, 203, 107683. https://doi.org/10.1016/j.appet.2024.107683

[3] Hint It Staff. (2025, February 9). Decision fatigue in menu design: How restaurants can improve customer choices. Hint-it Blog. `https://www.hint.it/blog/decision-fatigue-in-menu-design-how-restaurants-can-improve-customer-choices`

[4] Shin, B., Ryu, S., Kim, Y., & Kim, D. (2022). Analysis on review data of restaurants in Google Maps through text mining: Focusing on sentiment analysis. *Journal of Multimedia Information System*, 9(1), 61–68. https://doi.org/10.33851/JMIS.2022.9.1.61

[5] Zahrah, V. A., Nurdin, & Risawandi. (2024). Sentiment analysis of Google Maps user reviews on the Play Store using Support Vector Machine and Latent Dirichlet Allocation topic modeling. *International Journal of Engineering, Science and Information Technology*, 4(4), 87–100. https://doi.org/10.52088/ijesty.v4i4.580